

**ADVANCED LINKED LISTS EXERCISE
THIRTEENTH-WEEK ASSIGNMENT FOR THE DATA STRUCTURE**



Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026

Exercises

9.1 Evaluate the four basic linked list operations for a sorted doubly linked list and indicate the worst case time-complexities of each.

1. Traversal

```
1. current = head
2. while current is not None:
3.     print(current.data)
4.     current = current.next
```

Traversal in a sorted doubly linked list is performed by visiting each node one by one from the head node to the tail node. In the worst case, all nodes in the list must be visited to display all data. If the list contains n nodes, the traversal operation requires n steps, so the worst-case time complexity is $O(n)$.

2. Search

```
1. current = head
2. while current is not None and current.data != target:
3.     current = current.next
```

The search operation is performed by traversing the linked list until the target value is found. Since a linked list does not support direct index access like an array, the search must be done sequentially. In the worst case, the target value is located in the last node or does not exist in the list, so all nodes must be checked. Therefore, the worst-case time complexity for search is $O(n)$.

3. Insertion

```
1. while current is not None and current.data < value:
2.     current = current.next
```

In a sorted doubly linked list, insertion cannot be done randomly because the data order must remain sorted. The algorithm must first locate the correct position before inserting the new node. Although updating the pointers only requires constant time, finding the correct position may require traversing the entire list. Because of this, the worst-case time complexity for insertion is $O(n)$.

4. Deletion

```
1. while current is not None and current.data != target:
2.     current = current.next
```

The deletion operation begins by searching for the target node. After the target node is found, the previous and next pointers are updated so the node is removed from the linked list. Since locating the target node may require traversing the entire list, the worst-case time complexity for deletion is $O(n)$.

9.2 Evaluate the worst case time-complexity for the search operation on a doubly linked list using a probe pointer.

```
1. current = probe
```

```

2.
3. while current is not None and current.data != target:
4.     if target < current.data:
5.         current = current.prev
6.     else:
7.         current = current.next

```

A probe pointer is an additional reference used to remember the position of the previous search result in a doubly linked list. Instead of always starting the search from the head node, the next search operation can begin from the probe position. This technique can improve efficiency when multiple searches are performed on nearby values. For example, if the previous search stopped at value 58 and the next target value is 37, the algorithm can move backward from the probe node instead of starting from the beginning of the list. Likewise, if the target value is larger, the search can continue forward from the probe position. Although the probe pointer improves the average search performance, the worst-case situation still occurs when the target node is located far from the probe position or when the target value does not exist in the list. In such cases, the algorithm may still need to traverse almost all nodes in the linked list. Therefore, the worst-case time complexity for the search operation remains $O(n)$.

9.3 Evaluate the worst case time-complexity of the Sparse Matrix ADT implemented in Programming Project 9.7.

1. Traversal

```

1. current = rowHead
2.
3. while current is not None:
4.     print(current.value)
5.     current = current.nextCol

```

Traversal in a Sparse Matrix ADT is performed by visiting each non-zero element stored inside the multi-linked list structure. Since the matrix only stores non-zero values, the algorithm moves through the row links or column links instead of scanning every position in a normal matrix. In the worst case, the traversal operation must visit all non-zero elements in the sparse matrix. If there are n non-zero elements, then the traversal requires n steps. Therefore, the worst-case time complexity for traversal is $O(n)$.

2. Search

```

1. current = rowHead
2.
3. while current is not None and current.col != targetCol:
4.     current = current.nextCol

```

The search operation is used to locate a specific matrix element based on its row and column index. The algorithm traverses the linked structure until the target position is found. In the worst case, the target element may be located near the end of the row chain or may not exist at all. Because of this, the algorithm may need to examine every non-zero node in the structure. Therefore, the worst-case time complexity for search is $O(n)$.

3. Insertion

```
1. while current is not None and current.col < newCol:
2.     current = current.nextCol
```

Insertion in a Sparse Matrix ADT requires finding the correct row and column position before connecting the new node into both the row list and column list. The algorithm must preserve the linked structure correctly so traversal by rows and columns remains valid. Although updating the pointers only takes constant time, locating the correct insertion position may require traversing many nodes. In the worst case, the algorithm may need to scan all non-zero elements. Therefore, the worst-case time complexity for insertion is $O(n)$.

4. Deletion

```
1. while current is not None and current.col != targetCol:
2.     current = current.nextCol
```

The deletion operation begins by searching for the target node inside the sparse matrix structure. After the node is found, the links in both the row chain and the column chain must be updated so the node is removed safely. Since locating the target node may require traversing many non-zero elements, the deletion process may also require checking all nodes in the worst case. Therefore, the worst-case time complexity for deletion is $O(n)$.

9.4 Provide the code to implement the four basic operations — traversal, search, insertion, deletion — for an unsorted doubly linked list.

1. Traversal

```
1. current = head
2.
3. while current is not None:
4.     print(current.data)
5.     current = current.next
```

Traversal in an unsorted doubly linked list is performed by starting from the head node and visiting each node one by one until the end of the list is reached. This operation is commonly used to display all elements stored inside the linked list. Since the list is unsorted, every node must be visited sequentially. In the worst case, the traversal operation examines all nodes in the list, so the time complexity is $O(n)$.

2. Search

```
1. current = head
2.
3. while current is not None and current.data != target:
4.     current = current.next
```

The search operation is used to find a target value inside the doubly linked list. Because the list is unsorted, the algorithm cannot predict where the value is located. Therefore, the search must check each node sequentially until the target value is found. In the worst case, the target value may be stored in the last node or may not exist at all. As a result, all nodes may need to be examined, giving the search operation a worst-case time complexity of $O(n)$.

3. Insertion

```
1. newNode = Node(value)
2.
3. newNode.next = head
4. head.prev = newNode
5. head = newNode
```

Insertion in an unsorted doubly linked list is relatively simple because the new node can be inserted directly without maintaining any sorted order. One common approach is inserting the new node at the beginning of the list. The algorithm creates a new node, adjusts the pointers, and updates the head reference. Since no traversal is required when inserting at the front, the insertion operation can be completed efficiently. The worst-case time complexity for insertion at the beginning of the list is $O(1)$.

4. Deletion

```
1. current = head
2.
3. while current is not None and current.data != target:
4.     current = current.next
5.
6. if current is not None:
7.     current.prev.next = current.next
8.
9.     if current.next is not None:
10.        current.next.prev = current.prev
```

The deletion operation begins by searching for the target node inside the doubly linked list. Once the node is found, the previous node and next node are connected together so the target node is removed from the list. Because the list is unsorted, the algorithm may need to traverse many nodes before finding the target value. In the worst case, all nodes may need to be checked. Therefore, the worst-case time complexity for deletion is $O(n)$.

9.5 Provide the code to delete a node from a sorted doubly linked list.

```
1. current = head
2.
3. while current is not None and current.data < target:
4.     current = current.next
5.
6. if current is not None and current.data == target:
7.
8.     if current.prev is not None:
9.         current.prev.next = current.next
10.    else:
11.        head = current.next
12.
13.    if current.next is not None:
14.        current.next.prev = current.prev
```

The deletion operation in a sorted doubly linked list begins by searching for the target node. Since the list is sorted, the algorithm can stop early when it encounters a node

with a value greater than the target value. This makes the search process more efficient compared to an unsorted list. After the target node is found, the links between neighboring nodes must be updated. The previous node's next pointer is changed to point to the next node, while the next node's previous pointer is updated to point back to the previous node. This safely removes the target node from the linked list. The algorithm must also handle special cases carefully. If the node being deleted is the head node, the head pointer must be updated to the next node. If the node is the last node, the previous node simply points to `None`. Proper pointer management is important to maintain the structure of the doubly linked list after deletion. In the worst case, the algorithm may still need to traverse many nodes before finding the target value or determining that the value does not exist. Therefore, the worst-case time complexity for deletion is $O(n)$.

9.6 Provide the code to delete a node from a sorted singly linked circular list.

```
1. current = head
2. prev = None
3.
4. while True:
5.
6.     if current.data == target:
7.         break
8.
9.     prev = current
10.    current = current.next
11.
12.    if current == head:
13.        current = None
14.        break
15.
16. if current is not None:
17.
18.     if current == head:
19.
20.         tail = head
21.
22.         while tail.next != head:
23.             tail = tail.next
24.
25.         if head.next == head:
26.             head = None
27.         else:
28.             tail.next = head.next
29.             head = head.next
30.
31.     else:
32.         prev.next = current.next
```

The deletion operation in a sorted singly linked circular list begins by searching for the target node. Since the structure is circular, traversal continues until the algorithm either finds the target node or returns back to the head node. If the traversal reaches the head node again, it means the target value does not exist in the list. After the target node is found, the algorithm updates the links to remove the node from the circular list. If the node is located in the middle or at the end of the list, the predecessor node simply skips the target node and points directly to the next node. Special handling is required when deleting the head node. Because the list is circular, the last node must also be updated so it points to the new head node. If the list contains only one node, deleting that node will make the list empty and the head pointer becomes `None`. Since the algorithm may need to traverse the entire circular list before locating the target node, the worst-case time complexity for deletion is $O(n)$.

Programming Projects

9.1 Complete the implementation of the EditBuffer class from Section 9.5.

1. Constructor

```
1. def __init__(self):
2.     self._firstLine = _EditBufferNode(['\n'])
3.     self._lastLine = self._firstLine
4.     self._curLine = self._firstLine
5.     self._curLineNdx = 0
6.     self._curColNdx = 0
7.     self._numLines = 1
8.     self._insertMode = True
```

The EditBuffer class is used to simulate the internal structure of a simple text editor. The implementation uses a doubly linked list where each node stores one line of text inside a Python list. The constructor creates an empty buffer that initially contains one blank line with a newline character. The cursor starts at the first position of the first line, and the default mode is insert mode.

2. Cursor Movement

```
1. def moveLeft(self):
2.     if self._curColNdx == 0:
3.         if self._curLineNdx > 0:
4.             self.moveUp(1)
5.             self.moveLineEnd()
6.         else:
7.             self._curColNdx -= 1
```

Cursor movement operations are responsible for navigating through the text buffer. Methods such as `moveLeft()`, `moveRight()`, `moveUp()`, and `moveDown()` modify the current cursor position. When the cursor reaches the beginning or end of a line, it may move to another line automatically. The cursor is not allowed to move outside the document boundaries

3. Adding Characters

```
1. def addChar(self, char):
2.     if self.inInsertMode():
```

```

3.         self._curLine.text.insert(self._curColNdx, char)
4.     else:
5.         self._curLine.text[self._curColNdx] = char

```

The addChar() method inserts characters into the text buffer. In insert mode, characters are inserted and the remaining characters shift to the right. In overwrite mode, the existing character is replaced. If the inserted character is a newline character (\n), the current line is split into two lines using the breakLine() method.

4. Deleting Characters

```

1. def deleteChar(self):
2.     self._curLine.text.pop(self._curColNdx)

```

The deleteChar() method removes the character located at the current cursor position. If the deleted character is not a newline character, it is simply removed from the current line. However, if the deleted character is a newline character, the current line and the next line must be merged together.

5. Breaking a Line

```

1. def breakLine(self):
2.     newText = self._curLine.text[self._curColNdx:]

```

The breakLine() operation splits the current line into two separate lines at the cursor position. The characters after the cursor become part of the new line. The new node is inserted directly after the current node inside the doubly linked list. After the split operation, the cursor moves to the beginning of the new line.

9.2 Modify the moveLeft() and moveRight() methods to accept an integer to indicate the number of spaces to move in the respective direction.

1. moveLeft()

```

1. def moveLeft(self, steps):
2.
3.     for i in range(steps):
4.
5.         if self._curColNdx == 0:
6.
7.             if self._curLineNdx > 0:
8.                 self.moveUp(1)
9.                 self.moveLineEnd()
10.
11.         else:
12.             self._curColNdx -= 1

```

The modified moveLeft() method allows the cursor to move multiple positions to the left instead of moving only one position. A loop is used to repeat the movement operation according to the number of steps provided. If the cursor reaches the beginning of a line, it automatically moves to the end of the previous line.

2. moveRight()

```

1. def moveRight(self, steps):
2.

```

```

3.     for i in range(steps):
4.
5.         if self._curColNdx < self.numChars() - 1:
6.             self._curColNdx += 1
7.
8.         elif self._curLine.next is not None:
9.             self._curLine = self._curLine.next
10.            self._curLineNdx += 1
11.            self._curColNdx = 0

```

The modified `moveRight()` method moves the cursor multiple positions to the right. If the cursor reaches the end of the current line, it automatically continues to the beginning of the next line. The cursor movement stops when the end of the document is reached.

9.3 Modify the `EditBuffer` class to include the following operations: `getPage(first, last)`: Returns a vector of strings consisting of the rows `[first...last]`, which can be used for displaying a page of text. `insertString(str)`: Inserts a string of text within the current line starting at the current cursor position. `moveTo(lineNdx, colNdx)`: Moves the cursor to the indicated position within the buffer. If `lineNdx` is out of range, no action is taken. If `colNdx` is larger than the current line, then the cursor is positioned at the end of that line

1. `getPage(first, last)`

```

1. def getPage(self, first, last):
2.
3.     result = []
4.     current = self._firstLine
5.     index = 0
6.
7.     while current is not None:
8.
9.         if first <= index <= last:
10.            result.append("".join(current.text))
11.
12.            current = current.next
13.            index += 1
14.
15.     return result

```

The `getPage()` method returns multiple lines from the text buffer as a list of strings. This operation is useful for displaying part of the document, similar to how a text editor shows one page of text on the screen. The algorithm traverses the doubly linked list and collects the lines whose indexes are between `first` and `last`.

2. `insertString(str)`

```

1. def insertString(self, string):
2.
3.     for char in string:
4.         self.addChar(char)

```

The `insertString()` method inserts an entire string into the current cursor position. Instead of inserting characters manually one by one, the method automatically loops through

every character inside the string and inserts them sequentially using the `addChar()` method. If the string contains newline characters, the text buffer automatically creates new lines.

3. `moveTo(lineNdx, colNdx)`

```
1. def moveTo(self, lineNdx, colNdx):
2.
3.     if lineNdx < 0 or lineNdx >= self._numLines:
4.         return
5.
6.     current = self._firstLine
7.     index = 0
8.
9.     while index < lineNdx:
10.         current = current.next
11.         index += 1
12.
13.     self._curLine = current
14.     self._curLineNdx = lineNdx
15.
16.     if colNdx >= len(current.text):
17.         self._curColNdx = len(current.text) - 1
18.     else:
19.         self._curColNdx = colNdx
```

The `moveTo()` method moves the cursor directly to a specific line and column inside the text buffer. The algorithm first checks whether the target line exists. If the line index is invalid, no movement occurs. Otherwise, the linked list is traversed until the target line is reached. After moving to the correct line, the method checks the column index. If the requested column is larger than the line length, the cursor is placed at the end of the line. This operation allows direct navigation to any valid position inside the document.